Sprint 16 Structured Notes: Dates, Time, Audio, Video, and Media APIs (Sinhala Edition)

මෙම සටහනෙන් JavaScript මඟින් browser එකේ තිබෙන dates, time, audio, video, MIME types, codecs, සහ media streams වැනි පහසුකම් පාලනය කරන්නේ කොහොමද කියලා සරලව ඉගෙන ගනිමු. අපගේ ප්‍රධානම ඉලක්කය වන්නේ කිසිදු external library එකක් භාවිත නොකර, browser එකේම ඇති built-in APIs මඟින් මේවා සිදු කිරීමයි.

1. Browser APIs for Time and Media

Concept Explanation (Sinhala): JavaScript වලට core language එකක් විදිහට variables, arrays, objects, functions වැනි දේවල් තිබුණත්, browser එක ඇතුළෙදි අපිට වැඩ කරන්න තවත් විශේෂ built-in toolset එකක් ලැබෙනවා. මේවාට අපි **Browser APIs** කියලා කියනවා.

Sprint 16 වලදී අපි ඉගෙන ගන්නේ browser එකෙන් අපිට දෙන කාලය (time) සහ මාධ්‍ය (media) පාලනය කරන්න පුළුවන් APIs කිහිපයක් ගැනයි. උදාහරණයක් විදිහට current time එක ලබාගැනීම, audio files load කිරීම, play/pause කිරීම සහ browser එකෙන් යම් media format එකක් support කරනවාද කියා සෙවීම මේ හරහා සිදු කරන්න පුළුවන්.

Code Block:

```
const now = new Date();
console.log(now.getFullYear());

const audio = new Audio("sound.mp3");
audio.play();
```

Code Breakdown (Sinhala):

- `const now = new Date();` : මෙතනදී `Date` කියන built-in constructor එකෙන් අලුත් date object එකක් හදනවා. ඒකෙන් current date සහ time එක `now` කියන variable එකට ලැබෙනවා.
- `console.log(now.getFullYear());` : එම date object එකේ ඇති `getFullYear()` කියන method එකෙන් වත්මන් වර්ෂය (උදාහරණයක් විදිහට 2026) console එකේ print කරනවා.
- `const audio = new Audio("sound.mp3");` : browser එකේ පසුබිමෙන් "sound.mp3" කියන audio file එක load කරගන්න අලුත් `Audio` object එකක් නිර්මාණය කරනවා.
- `audio.play();` : එම audio object එකේ `play()` method එක කෝල් කිරීමෙන් සින්දුව හෝ ශබ්දය වාදනය කිරීම ආරම්භ කරනවා.

2. What Is a Date Object?

Concept Explanation (Sinhala): `Date` object එකක් කියන්නේ යම්කිසි නිශ්චිත මොහොතක (moment in time) දත්ත නියෝජනය කරන විශේෂ object එකක්.

මතක තබාගත යුතු වැදගත් රීතියක් තමයි, `Date` object එකක් කියන්නේ නිකන්ම text string එකක් නෙවෙයි. මේක ඇතුළේ දින සහ වේලාවන් format කරන්න සහ කියවන්න අවශ්‍ය methods රැසක් අඩංගු වෙනවා.

Code Block:

```
const now = new Date();
console.log(now);
```

Code Breakdown (Sinhala):

- `new Date()` මඟින් current date සහ time එක සහිත object එකක් සාදනවා.
- මෙය console එකේ print කළ විට `Wed May 06 2026 10:30:00 GMT+0530` වැනි output එකක් බලාගන්න පුළුවන්. මෙය ඔබේ පරිගණකයේ වෙලාව, browser එක සහ time zone එක මත වෙනස් වෙන්න පුළුවන්.

3. Creating Specific Dates (from Strings & Numbers)

Concept Explanation (Sinhala): අපිට නිශ්චිත දවසක් සඳහා `Date` object එකක් ක්‍රම දෙකකට හදාගන්න පුළුවන්:

1. **Using a String (date string එකක් භාවිතයෙන්):** string එකක් `new Date()` එකට pass කිරීමෙන්.
2. **Using Numbers (ඉලක්කම් භාවිතයෙන්):** year, month index, සහ day වෙන වෙනම numbers විදිහට pass කිරීමෙන්.

මෙහිදී ඉතාම වැදගත් කරුණක් තිබෙනවා: Numbers භාවිත කරන විට, JavaScript වල මාස (months) ගණන් කරන්නේ 0 ඉඳලා. ඒ කියන්නේ:

- 0 = January
- 1 = February
- 4 = May
- 11 = December

ඒ නිසා `new Date(2026, 4, 20)` කියන්නේ **May 20, 2026** මිසක් April 20, 2026 නෙවෙයි.

Code Block:

```
// ක්‍රමය 1: Date String එකක් භාවිතයෙන්
const birthdayFromString = new Date("2026-05-20");
console.log(birthdayFromString);

// ක්‍රමය 2: Numbers භාවිතයෙන් (Month index එක සිරෝ වලින් පටන් ගනී)
const birthdayFromNumbers = new Date(2026, 4, 20);
console.log(birthdayFromNumbers);
```

Code Breakdown (Sinhala):

- `"2026-05-20"` string එක කෙලින්ම ගත්තම standard format එකෙන් May 20 හඳුනාගන්නවා.
- `(2026, 4, 20)` වලදී, 4 වෙනි index එක May මාසය නිසා එයද May 20 ලෙස හඳුනාගන්නවා. පටලවා නොගැනීම සඳහා variable එකක් හදාගැනීම වඩා සුදුසුයි (උදා: `const may = 4;`).

4. Getting Parts of a Date

Concept Explanation (Sinhala): `Date` object එකක් ඇතුළෙන් අපිට අවශ්‍ය කොටස් (වර්ෂය, මාසය, දිනය, පැය, මිනිත්තු, තත්පර) වෙන වෙනම ලබාගැනීමට JavaScript වල methods රාශියක් තිබෙනවා.

ප්‍රධාන Methods ලැයිස්තුව:

- `getFullYear()` : වර්ෂය ලබාදේ (උදා: 2026).
- `getMonth()` : 0 සිට 11 දක්වා වන මාසයේ index එක ලබාදේ (May සඳහා 4 ලබාදේ).
- `getDate()` : මාසයේ දිනය ලබාදේ (1 සිට 31 දක්වා). **වැදගත්:** `getDate()` සහ `Date` object එක පටලවා ගන්න එපා. `getDate()` වලින් ලැබෙන්නේ දවසේ number එක විතරයි.
- `getHours()` : පැය ලබාදේ.
- `getMinutes()` : මිනිත්තු ලබාදේ.
- `getSeconds()` : තත්පර ලබාදේ.

Code Block:

```
const date = new Date(2026, 4, 20, 9, 30, 15);

console.log(date.getFullYear()); // Output: 2026
console.log(date.getMonth());    // Output: 4 (May)
console.log(date.getDate());     // Output: 20
console.log(date.getHours());    // Output: 9
console.log(date.getMinutes());  // Output: 30
console.log(date.getSeconds());  // Output: 15
```

Code Breakdown (Sinhala):

- මෙහිදී අපි May 20, 2026 දින උදේ 9:30:15 ට අදාළව `Date` object එකක් සකසා, එහි ඇති හැම කොටසක්ම අදාළ methods මඟින් console එකට ලබාගෙන තිබෙනවා.

5. Date.now() vs new Date() & Unix Epoch

Concept Explanation (Sinhala): මෙන්න තවත් නිතරම වරදින තැනක්:

- `new Date()` මඟින් ලැබෙන්නේ **Date object** එකක්.
- `Date.now()` මඟින් ලැබෙන්නේ **number** එකක් (timestamp එකක්).

මෙම number එකෙන් නියෝජනය කරන්නේ **Unix epoch** (ඒ කියන්නේ 1970 January 1 දින UTC මධ්‍යම රාත්‍රියේ) සිට මේ මොහොත දක්වා ගතවී ඇති මුළු මිලි තත්පර (milliseconds) ගණනයි. 1000 milliseconds $= 1$ second නිසා, timestamps දෙකක වෙනස බැලීමෙන් අපිට යම් කේතයක් (code) එකක් ක්‍රියාත්මක වීමට ගතවන කාලය මැනගන්න පුළුවන්.

Code Block:

```
const start = Date.now();

// කාලය ගතවන වැඩක් සිදු කිරීම (Loop එකක්)
for (let i = 0; i < 1000000; i++) {
  // Empty loop
}

const end = Date.now();
const difference = end - start;

console.log(`Time passed: ${difference} ms`);
```

Code Breakdown (Sinhala):

- `Date.now()` මඟින් වැඩේ පටන් ගන්න කොට තිබුණු timestamp එක `start` වලට ගන්නවා.
- Loop එක ඉවර වුණාම නැවත `Date.now()` මඟින් `end` timestamp එක ගන්නවා.
- `end - start` මඟින් එම loop එක දුවන්න ගතවුණු මුළු milliseconds ගණන console එකේ පෙන්වනවා.

6. UTC (Coordinated Universal Time)

Concept Explanation (Sinhala): UTC කියන්නේ ලෝකයේ හැමෝම පොදුවේ පිළිගන්නා සම්මත වේලාවයි (standard global time reference). අපේ පරිගණකයේ පෙන්වන්නේ local time zone එක වුවත්, standard ISO strings සාමාන්‍යයෙන් සකස් වෙන්නේ UTC පදනම් කරගෙනයි. `toISOString()` method එකෙන් ලැබෙන string එකේ අගට ඇති `Z` අකුරෙන් පෙන්වන්නේ එය UTC වෙලාව බවයි.

Code Block:

```
const now = new Date();
console.log(now.toISOString());
```

Code Breakdown (Sinhala):

- `now.toISOString()` මඟින් `2026-05-06T09:30:00.000Z` ආකාරයේ machine-friendly string එකක් ලබාදෙනවා. මෙය දත්ත base එකක සේව් කරන්න හෝ APIs වලට යවන්න ඉතාම සුදුසුයි.

7. Formatting Dates for Users

Concept Explanation (Sinhala): පරිශීලකයින්ට (users) කියවන්න ලේසි විදිහට dates format කරන්න ක්‍රම 2ක් තිබෙනවා:

1. `toLocaleDateString()` : සරලවම locale එකට අනුව දවස format කරලා දෙනවා (උදා: `5/6/2026`).
2. `Intl.DateTimeFormat` : වඩාත් පාලනයක් සහිතව, නැවත නැවත භාවිත කළ හැකි (reusable) formatters හදන්න පුළුවන්.

Code Block:

```
const date = new Date(2026, 4, 20);

// ක්‍රමය 1: toLocaleDateString
console.log(date.toLocaleDateString("en-US", {
  year: "numeric",
  month: "long",
  day: "numeric"
})); // Output: May 20, 2026

// ක්‍රමය 2: Intl.DateTimeFormat
const formatter = new Intl.DateTimeFormat("en-US", {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric"
});
console.log(formatter.format(date)); // Output: Wednesday, May 20, 2026
```

Code Breakdown (Sinhala):

- `toLocaleDateString` එකට options pass කිරීමෙන් "May 20, 2026" ලෙස ලස්සනට පෙන්වන්න පුළුවන්.
- `Intl.DateTimeFormat` මඟින් `formatter` කියන reusable object එක හදාගන්නවා. පසුව `formatter.format(date)` මඟින් ඕනෑම date එකක් එම රටාවටම format කරගත හැකියි.

8. Date Mini-Function: Format a Due Date

Concept Explanation (Sinhala): දැන් අපි ඉගෙන ගත්තු දේවල් එකතු කරලා, Task Management App එකක වැඩක් අවසන් කළ යුතු දිනය (Due Date) ලස්සනට format කරලා පෙන්වන සැබෑ ප්‍රායෝගික function එකක් හදමු.

Code Block:

```
function formatDueDate(year, monthIndex, day) {
  const dueDate = new Date(year, monthIndex, day);
  const formatter = new Intl.DateTimeFormat("en-US", {
    dateStyle: "medium"
  });
  return formatter.format(dueDate);
}

console.log(formatDueDate(2026, 4, 20)); // Output: May 20, 2026
```

Code Breakdown (Sinhala):

- `formatDueDate(year, monthIndex, day)` : පරාමිතීන් (parameters) 3ක් ගන්නා function එකක් ලියනවා.
- `new Date(year, monthIndex, day)` : එම අගයන්ගෙන් date object එකක් හදනවා.
- `Intl.DateTimeFormat("en-US", { dateStyle: "medium" })` : මධ්‍යස්ථ ප්‍රමාණයේ දිනයක් පෙන්වන formatter එකක් සකසනවා.
- `return formatter.format(dueDate)` : format කරගත් text එක නැවත ලබාදෙනවා (return කරනවා).

9. Audio in JavaScript & HTMLAudioElement

Concept Explanation (Sinhala): JavaScript වලට browser එක ඇතුළේ ශබ්ද (audio) පාලනය කිරීමේ හැකියාව තිබෙනවා. මෙහිදී සලකා බැලිය යුතු ප්‍රධාන කරුණු කිහිපයකි:

- `new Audio("path.mp3")` : මඟින් backend එකේ ක්‍රියාත්මක වන `HTMLAudioElement` object එකක් සෑදේ.
- `play()` & `pause()` : වාදනය කිරීමට සහ නැවැත්වීමට භාවිත කරන ක්‍රමවේද වේ.
- **Autoplay Restriction (ස්වයංක්‍රීයව වාදනය වැළැක්වීම):** පරිශීලකයා පිටුව සමඟ කිසිදු ගනුදෙනුවක් (click එකක් වැනි user gesture එකක්) කිරීමට පෙර සින්දු play කිරීම browsers විසින් block කරනු ලබයි. එම නිසා සැමවිටම `play()` ක්‍රියාත්මක කළ යුත්තේ click event listener එකක් ඇතුළතයි.

Code Block:

```
const audio = new Audio("sound.mp3");
const playButton = document.getElementById("playButton");

playButton.addEventListener("click", () => {
  audio.play();
});
```

Code Breakdown (Sinhala):

- පිටුව load වන විටම සින්දුව පටන් ගන්නේ නැතිව, පරිශීලකයා `playButton` එක click කළ විට පමණක් `audio.play()` ක්‍රියාත්මක වන පරිදි ආරක්ෂිතව සකසා ඇත.

10. Controlling and Resetting Audio

Concept Explanation (Sinhala): JavaScript audio object එකක කෙලින්ම සින්දුවක් මුලටම නැවැත්වීමට `stop()` කියලා method එකක් නැත. නමුත් සින්දුවක් stop කර මුලටම ගැනීමට (reset කිරීමට) අපිට පහත උපක්‍රමය භාවිත කළ හැකියි:

1. `pause()` එකෙන් සින්දුව නවත්වනවා.
2. `currentTime = 0` මඟින් සින්දුව වාදනය වන වෙලාව (playback position) නැවත මුලටම (තත්පර 0 ට) රැගෙන යනවා.

Code Block:

```
const audio = new Audio("sound.mp3");

// Pause කිරීම
audio.pause();

// Reset කර මුලටම ගැනීම (Stop කිරීම)
audio.pause();
audio.currentTime = 0;

// සින්දුව මැදකට (තත්පර 30 ට) පැනීම
audio.currentTime = 30;
```

Code Breakdown (Sinhala):

- `currentTime` property එකෙන් අපිට සින්දුව වාදනය වන වෙලාව තත්පර වලින් කියවන්නත්, අපිට අවශ්‍ය තත්පරයකට සින්දුව ඉදිරියට හෝ පසුපසට ධාවනය කරවන්නත් පුළුවන්.

11. canPlayType() Method

Concept Explanation (Sinhala): සෑම browser එකක්ම එක හා සමානව හැම audio/video file formats (උදා: .mp3, .ogg, .webm) සපෝට් කරන්නේ නැත. අපිට යම් media file එකක් play කරන්න කලින්, browser එකට ඒක ප්ලේ කරන්න පුළුවන්ද කියලා අහලා බලන්න `canPlayType()` method එක භාවිත කරන්න පුළුවන්.

මෙයට අදාළව browser එකෙන් දෙන පිළිතුරු වර්ග 3කි:

- `"probably"` : අනිවාර්යයෙන්ම වගේ ප්ලේ කරන්න පුළුවන්.
- `"maybe"` : සමහරවිට ප්ලේ කරන්න පුළුවන් වෙයි.
- `""` (empty string): ප්ලේ කරන්න බැහැ.

Code Block:

```
const audio = new Audio();
const supportStatus = audio.canPlayType("audio/mpeg");

if (supportStatus === "") {
  console.log("This browser does not support MP3 files.");
} else {
  console.log("This browser can probably/maybe play MP3 files.");
}
```

Code Breakdown (Sinhala):

- `audio.canPlayType("audio/mpeg")` මඟින් "audio/mpeg" (MP3 වල MIME type එක) සපෝට් කරනවාදැයි පරීක්ෂා කර, එය හිස් string එකක් නොවේ නම් support කරන බව හඳුනා ගනී.

12. Media Formats, MIME Types, and Codecs

Concept Explanation (Sinhala): media files සමඟ වැඩ කිරීමේදී මෙම වචන 3 නිතරම අසන්නට ලැබේ. මේවා සරලව තේරුම් ගනිමු:

- **Media Format (Container):** මේක හරියට පෙට්ටියක් (box) වගේ. සින්දුව හෝ වීඩියෝව අසුරා ඇති file format එක (උදා: `.mp3` , `.mp4` , `.webm`).
- **Codec:** පෙට්ටිය ඇතුළේ තියෙන audio/video දත්ත compress කරලා තියෙන සහ කියවන ක්‍රමය (algorithm) තමයි codec එක කියන්නේ. එකම `.mp4` පෙට්ටිය ඇතුළේ වෙනස් codecs තියෙන්න පුළුවන්.
- **MIME Type:** මේක browser එකට පෙට්ටිය ඇරලා බලන්නේ නැතුව ඇතුළේ තියෙන්නේ මොන වගේ දෙයක්ද කියලා අඳුරගන්න ගහපු ලේබල් එකක් (label) වගේ (උදා: `audio/mpeg` , `video/mp4`).

ප්‍රධාන MIME Types සටහන:

File Type	Extension	MIME Type
MP3 Audio	`.mp3`	`audio/mpeg`
MP4 Audio	`.mp4`	`audio/mp4`
MP4 Video	`.mp4`	`video/mp4`
WebM Video	`.webm`	`video/webm`
Ogg Audio	`.ogg`	`audio/ogg`

Code Block:

```
<!-- HTML වලදී multiple sources භාවිත කරමින් browser එකට ගැලපෙන දේ තෝරා
ගැනීමට ඉඩදීම -->
<audio controls>
  <source src="sound.mp3" type="audio/mpeg">
  <source src="sound.ogg" type="audio/ogg">
  Your browser does not support audio elements.
</audio>
```

Code Breakdown (Sinhala):

- `<source>` elements කිහිපයක් දැමීමෙන්, browser එකට මුලින්ම ඇති MP3 file එක ප්ලේ කිරීමට නොහැකි නම් ඊළඟට ඇති Ogg file එක ප්ලේ කිරීමට උත්සාහ කරයි. කිසිවක් බැරි නම් fallback text එක පෙන්වයි.
- `controls` attribute එක දැමීමෙන් play, pause, volume සහ seek bar එක සහිත built-in player interface එක පරිශීලකයාට පෙන්වයි.

13. HTMLMediaElement & Shared Interface

Concept Explanation (Sinhala): HTML වල තිබෙන `<audio>` සහ `<video>` කියන elements දෙකම නිර්මාණය වෙලා තියෙන්නේ `HTMLMediaElement` කියන පොදු base interface එක මතයි. ඒ නිසා, මේ දෙවර්ගයටම පොදුවේ පාලනය කළ හැකි සර්වසම properties සහ methods සමූහයක් පවතී:

- `play()` / `pause()`
- `currentTime`
- `src`
- `muted` (true / false ලෙස sound mute කිරීමට)
- `volume` (0.0 සිට 1.0 දක්වා sound අඩු වැඩි කිරීමට)

Code Block:

```
const player = document.getElementById("myVideoPlayer");

player.volume = 0.5; // සින්දුවේ සද්දේ බාගයක් (50%) දක්වා අඩු කිරීම
player.muted = true; // සද්දේ සම්පූර්ණයෙන්ම නැති කිරීම
player.currentTime = 10; // විඩියෝ එක තත්පර 10 වෙනි තැනට රැගෙන යාම
```

Code Breakdown (Sinhala):

- HTML වල ඇති video player එකක් JavaScript මඟින් select කරගෙන, එහි පරිමාව (volume) සහ ධාවන කාලය (currentTime) වෙනස් කරන ආකාරය මෙහි පෙන්වා ඇත.

14. Media Streams & Camera/Microphone Permissions bascs

Concept Explanation (Sinhala): Media Stream එකක් කියන්නේ සේව් කරලා තියෙන file එකක් නෙවෙයි, camera එකකින් හෝ microphone එකකින් සජීවීව (live) ලැබෙන දත්ත ගලායාමකටයි. කැමරා හෝ මයික්‍රෆෝන දත්ත ඉතා සංවේදී (sensitive) නිසා, browser එක කිසිවිටෙකත් පරිශීලකයාගේ අවසරයකින් තොරව සජීවී දත්ත ලබා නොදේ. `navigator.mediaDevices.getUserMedia()` මඟින් අවසරය ඉල්ලා සිටින අතර, පරිශීලකයා එය "Allow" කළහොත් පමණක් අපට live stream එක ලැබෙනු ඇත.

Code Block:

```
async function startCamera() {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: true,
      audio: false
    });

    const videoElement = document.getElementById("preview");
    videoElement.srcObject = stream;
  } catch (error) {
    console.log("Camera access denied or error occurred: ", error);
  }
}
```

Code Breakdown (Sinhala):

- `navigator.mediaDevices.getUserMedia({ video: true, audio: false })` : browser එකෙන් කැමරාව පමණක් භාවිත කිරීමට අවසර ඉල්ලන පොප්-අප් (permission pop-up) එකක් පෙන්වයි.
- `videoElement.srcObject = stream` : පරිශීලකයා අවසර දුන් පසු ලැබෙන සජීවී stream එක HTML `<video>` element එකට සම්බන්ධ කර සජීවී දර්ශන පෙළක් (live camera preview) පෙන්වයි.

15. Comprehensive Practice Mini-Build

Concept Explanation (Sinhala): දැන් අපි ඉගෙන ගත්තු සියලුම සිද්ධාන්ත එකතු කරලා එකම HTML පිටුවක් ඇතුළේ වැඩ කරන ප්‍රායෝගික වැඩසටහනක් නිර්මාණය කරමු. මෙහිදී:

1. Due Date Formatter එකක් (දිනය ලස්සනට සකසන කොටසක්) සහ
2. සැබෑ Play, Pause, සහ Restart බොත්තම් සහිත Audio Controller එකක් අඩංගු වේ.

Code Block:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sprint 16 Practice Project</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
      background-color: #f4f4f9;
    }
    section {
      background: white;
      padding: 20px;
      margin-bottom: 20px;
      border-radius: 8px;
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }
    button {
      padding: 10px 15px;
      font-size: 14px;
      border: none;
      background-color: #007bff;
      color: white;
      border-radius: 4px;
      cursor: pointer;
      margin-right: 10px;
    }
    button:hover {
      background-color: #0056b3;
    }
    .status {
      font-style: italic;
      color: #555;
    }
  </style>
</head>
<body>

  <h1>Sprint 16 Practice Dashboard</h1>

  <!-- 1 කොටස: Due Date Formatter -->
  <section>
    <h2>Due Date Formatter</h2>
    <p id="dueDateOutput">Due date will appear here.</p>
    <button id="showDueDateButton">Show Due Date</button>
  </section>

  <!-- 2 කොටස: Custom Audio Controls -->
  <section>
    <h2>Audio Controls</h2>
    <button id="playButton">Play</button>
    <button id="pauseButton">Pause</button>
    <button id="restartButton">Restart</button>
    <p id="audioStatus" class="status">Audio status: Not started.</p>
  </section>

  <script>
    // --- DUE DATE SECTION ---
    const dueDateOutput = document.getElementById("dueDateOutput");
    const showDueDateButton =
document.getElementById("showDueDateButton");

    function formatDueDate(year, monthIndex, day) {
      const dueDate = new Date(year, monthIndex, day);
      const formatter = new Intl.DateTimeFormat("en-US", {
        dateStyle: "full"
      });
      return formatter.format(dueDate);
    }

    showDueDateButton.addEventListener("click", () => {
      // 4 කියන්නේ May (Zero-based month index)
      const formattedDate = formatDueDate(2026, 4, 20);
      dueDateOutput.textContent = `Task is due on ${formattedDate}.`;
    });

    // --- AUDIO SECTION ---
    // පසුබිමෙන් ප්ලේ වීමට online free mp3 sound එකක් හෝ sound.mp3 යොදන්න
    const audio = new Audio("
[https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3]
(https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3)");

    const playButton = document.getElementById("playButton");
    const pauseButton = document.getElementById("pauseButton");
    const restartButton = document.getElementById("restartButton");
    const audioStatus = document.getElementById("audioStatus");

    playButton.addEventListener("click", () => {
      audio.play();
      audioStatus.textContent = "Audio status: Playing music stream...";
    });

    pauseButton.addEventListener("click", () => {
      audio.pause();
      audioStatus.textContent = "Audio status: Paused.";
    });

    restartButton.addEventListener("click", () => {
      audio.pause();
      audio.currentTime = 0; // මුලටම Reset කිරීම
      audio.play();
      audioStatus.textContent = "Audio status: Restarted from the
beginning.";
    });
  </script>

</body>
</html>
```

Code Breakdown (Sinhala):

- `formatDueDate(2026, 4, 20)` : බොත්තම ක්ලික් කළ විට May 20, 2026 දිනය "Wednesday, May 20, 2026" ලෙස ලස්සනට textContent එකට ලබාදේ.
- `audio.currentTime = 0` : "Restart" බොත්තම ක්ලික් කළ විට සින්දුව මුලින්ම නවතා, ධාවන කාලය බිංදුව කර නැවත මුල සිට වාදනය (play) කරවයි.

16. Common Debug Challenges (නිතරම වරදින තැන් සහ විසඳුම්)

Concept Explanation (Sinhala): සංවර්ධකයෙකු (developer) ලෙස වැඩ කරන විට ඔබට මුහුණ දීමට සිදුවන ප්‍රධාන ගැටලු 4ක් සහ ඒවා විසඳන නිවැරදි ආකාරය පහත දැක්වේ.

Challenge 1: වැරදි මාසය ලැබීම

- **වැරදි කේතය:** `const date = new Date(2026, 5, 20);` (May 20 බලාපොරොත්තුවෙන් ලිවීම)
- **ගැටලුව:** Month index 5 යනු June මාසයයි.
- **නිවැරදි කේතය:** `const date = new Date(2026, 4, 20);` (May සඳහා 4 යොදන්න)

Challenge 2: Page Load එකේදීම Audio Play නොවී Block වීම

- **වැරදි කේතය:** `const audio = new Audio("sound.mp3"); audio.play();` (නිකන්ම script එක ඇතුළේ ලිවීම)
- **ගැටලුව:** පරිශීලකයා කිසිදු click එකක් කර නොමැති නිසා browser එක මඟින් autoplay block කරනු ලබයි.
- **නිවැරදි කේතය:** සැමවිටම `button.addEventListener("click", () => { audio.play(); });` වැනි user event එකක් ඇතුළත වාදනය කරන්න.

Challenge 3: Restart කළ විට සින්දුව මුලට නොයෑම

- **වැරදි කේතය:** `restartBtn.addEventListener("click", () => { audio.play(); });` (නිකන්ම play කිරීම)
- **ගැටලුව:** මෙය සින්දුව නවත්වන්නේ හෝ මුලට ගෙන යන්නේ නැත.
- **නිවැරදි කේතය:** `audio.pause(); audio.currentTime = 0; audio.play();` ලෙස ලියන්න.

Challenge 4: වැරදි MIME Type භාවිතය

- **වැරදි කේතය:** `<source src="sound.mp3" type="video/mp4">`
- **ගැටලුව:** සින්දුවක් (audio) සඳහා video MIME type එකක් ලබාදී තිබීම.
- **නිවැරදි කේතය:** `<source src="sound.mp3" type="audio/mpeg">` ලෙස නිවැරදි වර්ගය ලබාදෙන්න.

💡 Quick Recall Sheet (කෙටි මතක ආධාරකය)

- `new Date()` → වත්මන් දිනය සහ වේලාව සහිත Object එකක් සාදයි.
- `Date.now()` → Unix epoch (1970 Jan 1) සිට ගතවූ මුළු milliseconds ගණන (number එකක්) ලබාදේ.
- `getMonth()` → 0 සිට 11 දක්වා වන මාසයේ දර්ශකය (0-based index) ලබාදේ. (January = 0).
- `getDate()` → දවසේ අංකය (1-31) ලබාදේ.
- `toLocaleDateString()` → කියවීමට පහසු දේශීය දින ආකෘතියක් සාදයි.
- `new Audio()` → JavaScript මඟින් audio පාලනය කළ හැකි `HTMLAudioElement` එකක් සාදයි.
- `audio.pause(); audio.currentTime = 0;` → සින්දුවක් නැවැත්වීමට (stop / reset කිරීමට) ඇති එකම ක්‍රමයයි.
- **MIME Type** → ගොනුවේ අඩංගු මාධ්‍ය වර්ගය හඳුනා ගැනීමට බ්‍රවුසරයට දෙන ලේබලයයි (උදා: `audio/mpeg`).
- **Media Stream** → කැමරාවෙන් හෝ මයික්‍රෆෝනයෙන් ලැබෙන සජීවී දත්ත වන අතර, මේ සඳහා අනිවාර්යයෙන්ම පරිශීලකයාගේ අවසරය (permission) අවශ්‍ය වේ.